CASE STUDY 03 · IN ACTIVE DEVELOPMENT

"Mac"
A voice assistant that does things

An always-on assistant for my MacBook that wakes on its name, talks back in natural Urdu or English, and actually controls the laptop — with spoken confirmation before anything destructive.

ROLE	PLATFORM	BRAIN
Sole developer	macOS (Apple Silicon)	Gemini Live · Ollama fallback

LANGUAGES	STATUS
Urdu & English	Working daily build

01 / the goal

The goal

Voice assistants rarely understand Pakistani Urdu, and they mostly answer questions rather than *doing* things. I wanted an assistant that feels like talking to a person — fast, natural turn-taking, replies in the language I spoke — and can carry out real tasks on my laptop for free.

02 / what it does

What it does

Wake word

Always listening in the background for its name, "Mac" — it answers "جی زید؟" and is ready.

Natural conversation

Follow-up questions without repeating the wake word, with memory of recent context.

Two languages

Replies in Urdu when spoken to in Urdu, English when spoken to in English.

Laptop control

Opens apps, plays YouTube videos, checks the weather, takes screenshots, changes volume.

WhatsApp

Sends messages to contacts by name, then reports back "ہو گیا".

Safe by design

Restart, shutdown and deleting files need a spoken "jee" before anything happens.

03 / architecture — three versions

Architecture — three versions

v1 · Fully local

Speech-to-text
Local transcription

→

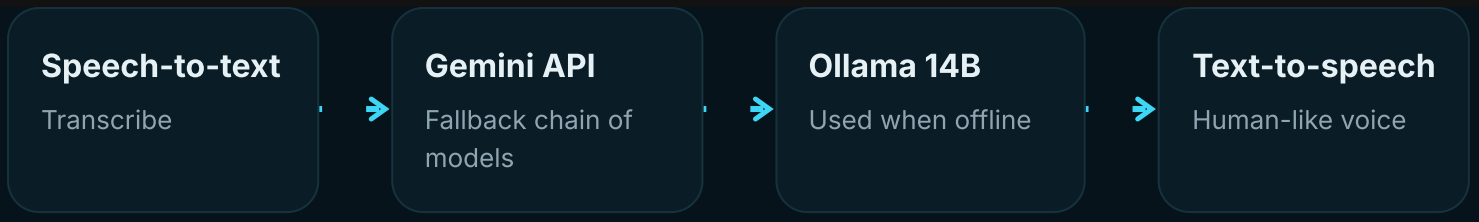
Ollama · Qwen 2.5
Local LLM on the M2 Pro

→

Text-to-speech
Spoken reply

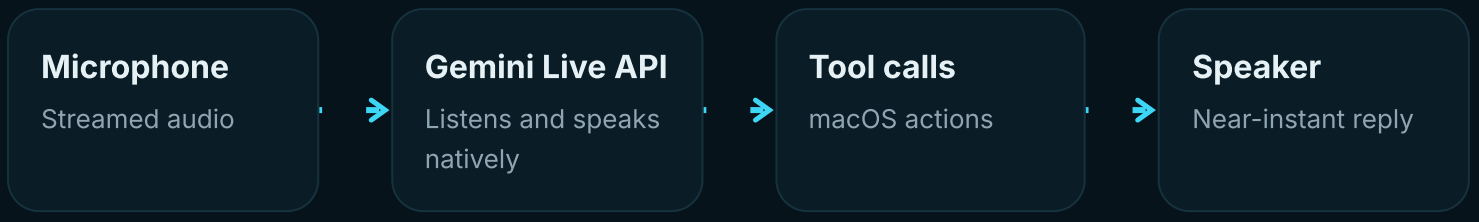
Free and private, but the 7B model produced broken, repetitive Urdu.

v2 · Cloud first, local fallback



Much better Urdu, but three stages in a row (transcribe → think → speak) were too slow for natural conversation.

v3 · Real-time native audio



Gemini Live hears and speaks audio directly, removing two of the three stages. The local Qwen 2.5 14B model stays as the offline fallback.

04 / challenges

Challenges

Urdu quality

The small local model repeated itself. Moving to the 14B model and tuning generation (a repeat penalty and a token cap) fixed the loops.

Overloaded models and timeouts

Free-tier models sometimes returned "high demand" errors. I built an ordered fallback chain. Setting the API timeout too low made the server reject requests, so the server timeout stays at 10 s while a shorter app-level timeout — a background thread and a queue — moves on to the next model.

Hearing itself

The assistant could mishear its own speech, or background noise in a busy shop, as the wake word. I tightened when it listens and how it confirms it was addressed.

macOS permissions

Controlling apps needs Accessibility, Automation, Screen Recording and Full Disk Access. I learned what each permission unlocks and how to tell a permission problem from an API problem.

Finding the right contact

Spoken names don't match contact names exactly; matching had to be forgiving without ever messaging the wrong person.

05 / what i learned

What I learned

- Latency is a product feature: architecture choices decide whether a conversation feels human.
- Designing failover between cloud and local models.
- Giving an AI real power over a computer safely — confirmations for anything destructive.

06 / documentation

Documentation

Requirements

macOS on Apple Silicon, Python 3.11, Homebrew, ffmpeg, and Ollama with `qwen2.5:14b` pulled for offline use. A free Gemini API key for online mode.

Setup

1. `ollama pull qwen2.5:14b`
2. Create a virtual environment and install the Python requirements.
3. Add the Gemini key to the environment.
4. Grant the terminal Accessibility, Automation, Screen Recording and Full Disk Access in System Settings.
5. Run the assistant; it starts in the background and waits for "Mac".